Curso: R-Evolution: Análisis de Datos con R+IA

Néstor Montaño Pulzara

ESPOL

2026

Nota:
Con *Alt* + *F* o *Option* + *F* puede hacer que estas diapositivas ocupen todo el navegador

Curso: R-Evolution: Análisis de Datos con R+IA

de Cero a Resultados

Antes de comenzar...

Una advertencia honesta

Un médico aprende anatomía antes de operar.
Un arquitecto aprende estática antes de diseñar.
Un piloto aprende instrumentos antes de volar.

Cada concepto de estas primeras clases aparecerá de nuevo — y tendrá sentido — cuando analices datos reales.
La base no es un obstáculo: **es un atajo**.

Como en los **fundamentos en cualquier disciplina**, las **primera parte** se sienten así:

"¿Para qué sirve todo esto? ¿Cuándo vamos a hacer algo útil?"

Y es completamente normal sentirlo.

Debemos primero saber instalar, configurar; entender que son los objetos, tipos de datos, operadores... cosas que por sí solas no resuelven ningún problema de negocio.

Confía en el proceso. La utilidad práctica llega, y cuando llega, llega rápido.

Introducción a R y RStudio

Curso: R-Evolution: Análisis de Datos con R+IA

Néstor Montaño P.

¿Por qué R?

- Software Libre (Open Source), gratuito y de desarrollo independiente,
- Uno de los lenguajes más usados para Data Science,
- Posee uno de los mejores sistemas gráficos existentes,
- Gran y creciente cantidad de usuarios y desarrolladores,
- CRAN: +20.000 paquetes disponibles y aparte hay paquetes instalables desde GitHub,
- Curva de aprendizaje simple (desde que apareció el tidyverse),
- Flexible y se integra bien con otros sistemas u aplicaciones: Bases de datos, software de inteligencia de negocios, etc.
- Rico ecosistema de paquetes libres y pagados que extienden las capacidades de R
- La mayoría de los nuevos métodos estadísticos se desarrollan primero en R



Comunidad R

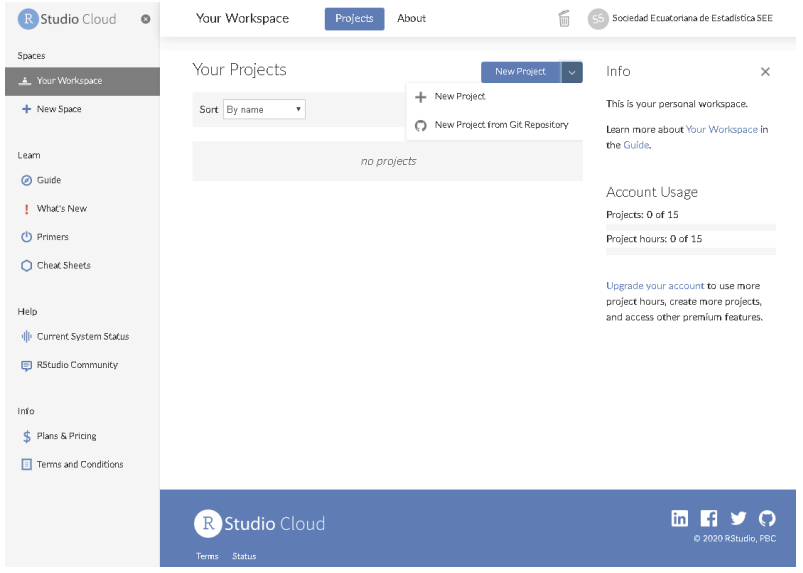
fuente: Open Data Science - Medium

RStudio / Posit Cloud

Otra forma de interactuar con R es usar **Posit Cloud** directamente desde el navegador:

👉 https://login.rstudio.cloud

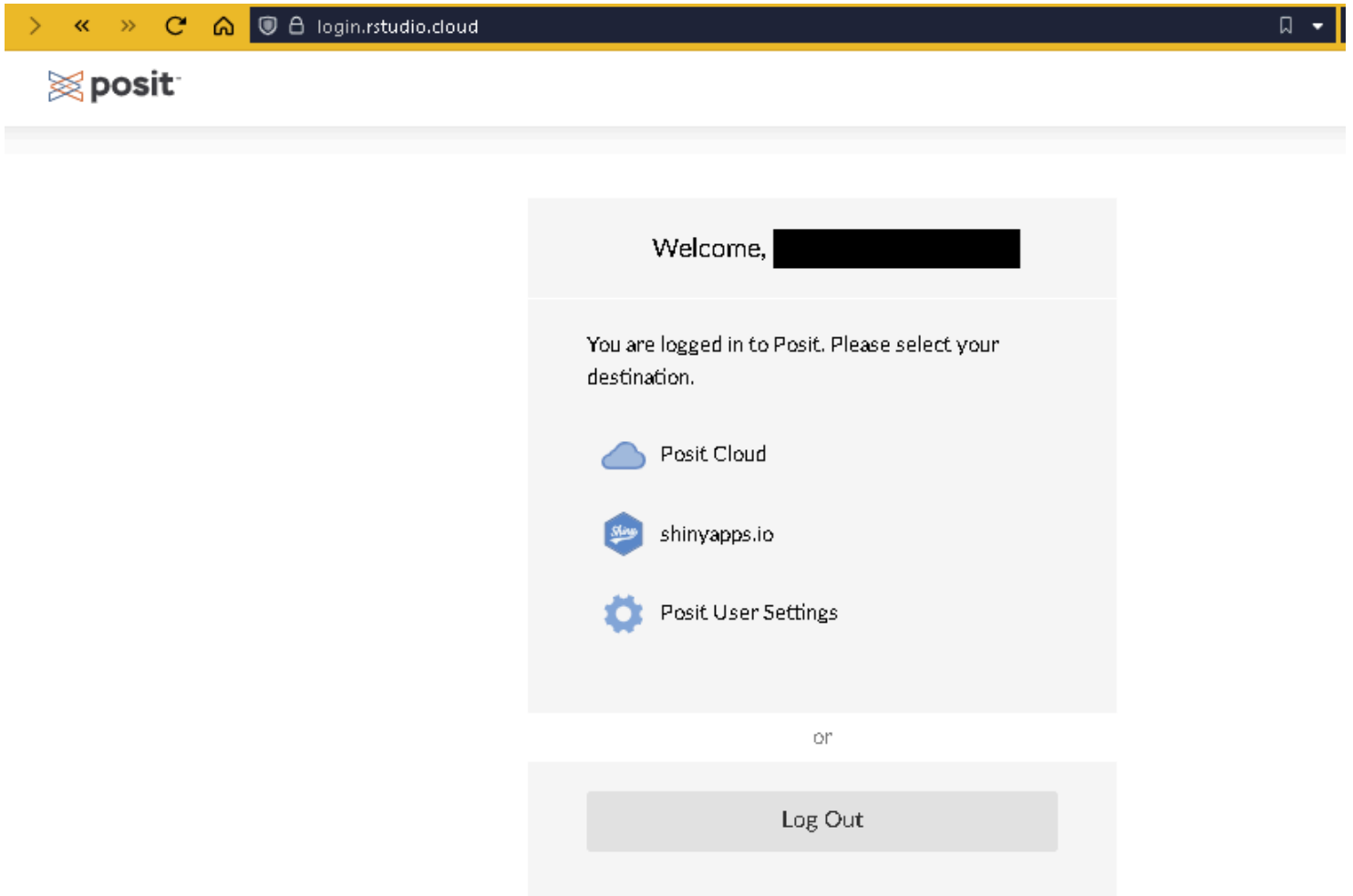
Ideal para comenzar sin instalar nada localmente.



RStudio Cloud

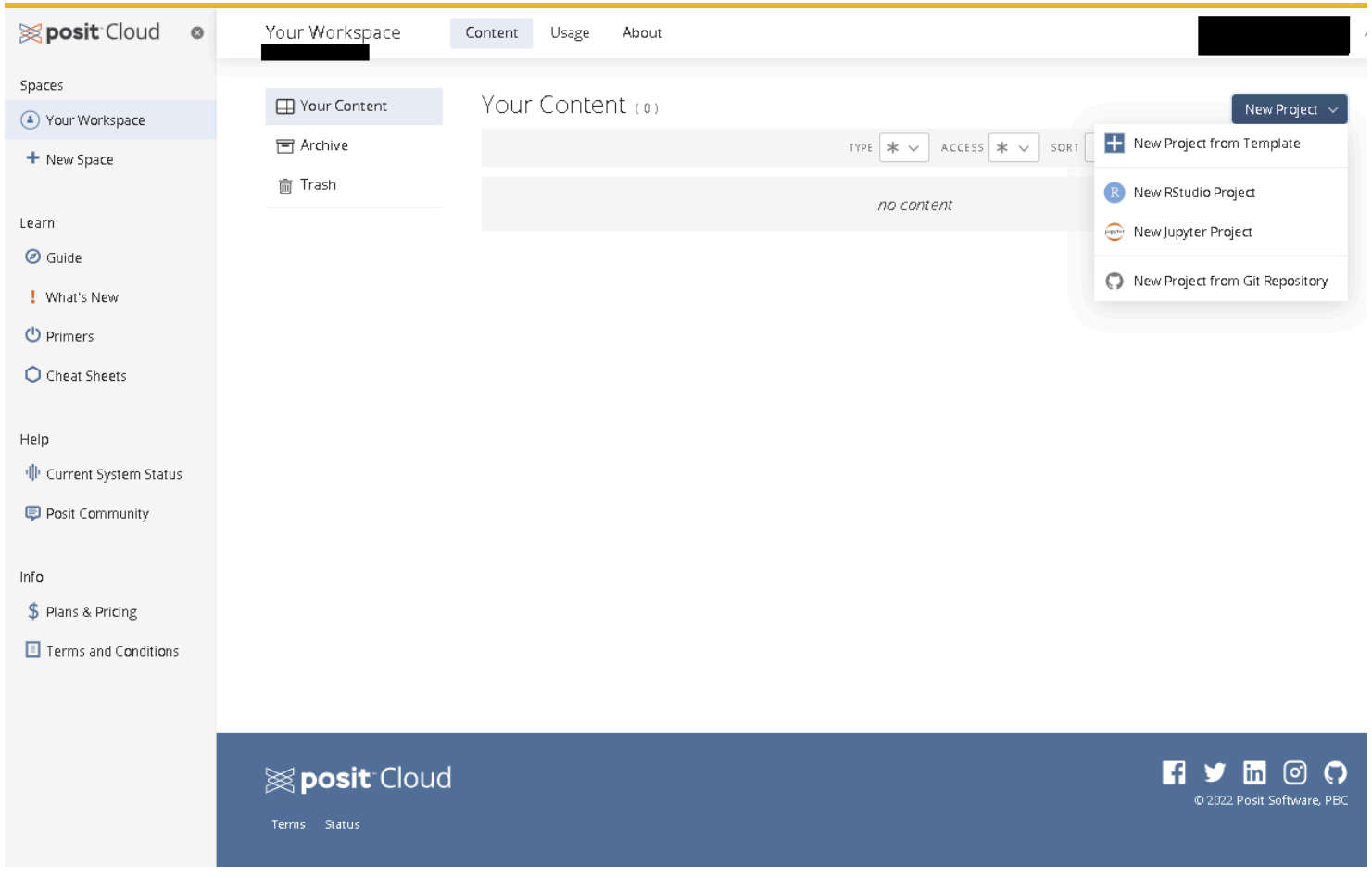
RStudio Cloud

Iniciar sesion en Posit Cloud



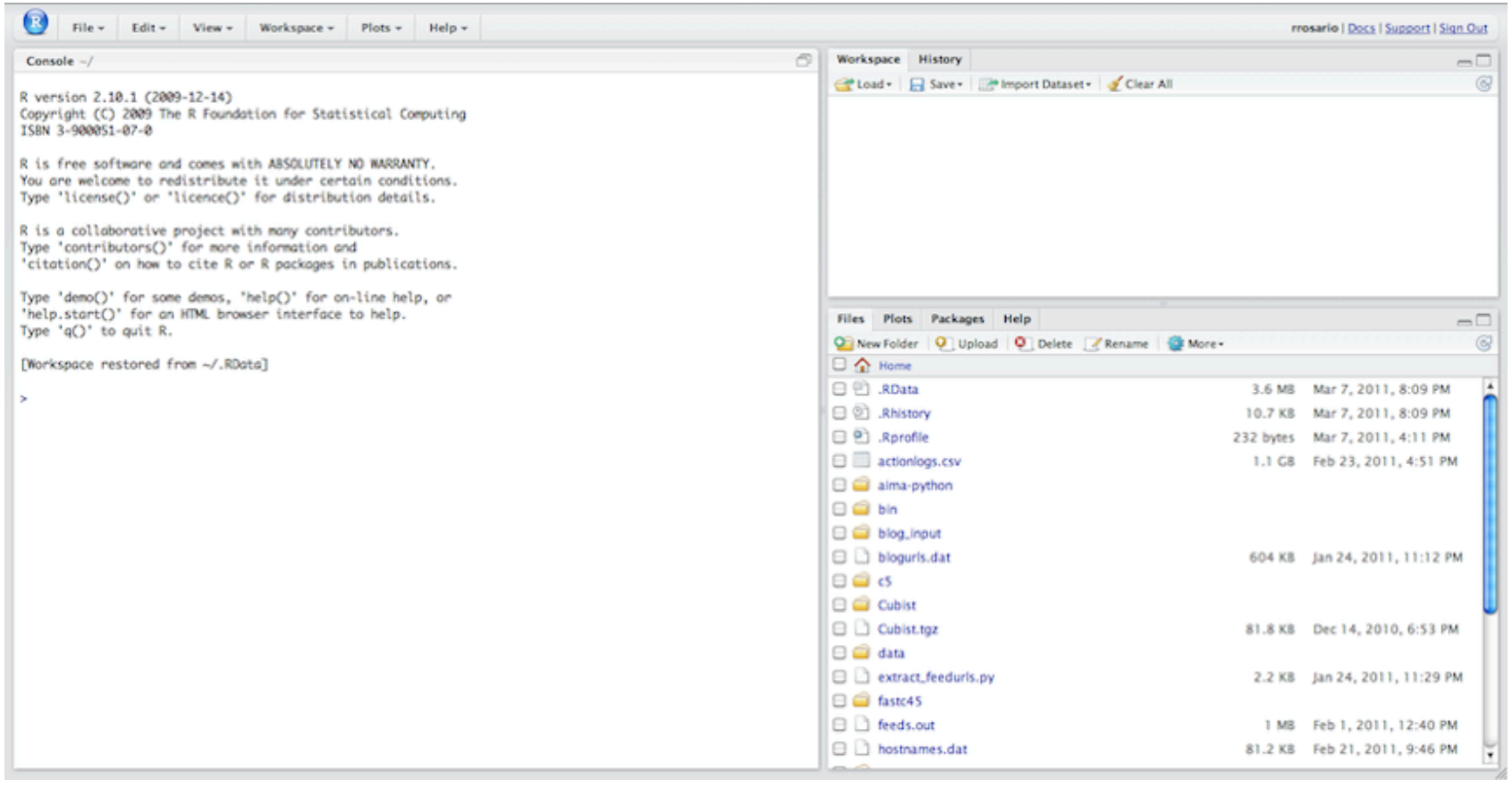
RStudio Cloud

Iniciar un Proyecto en Posit Cloud



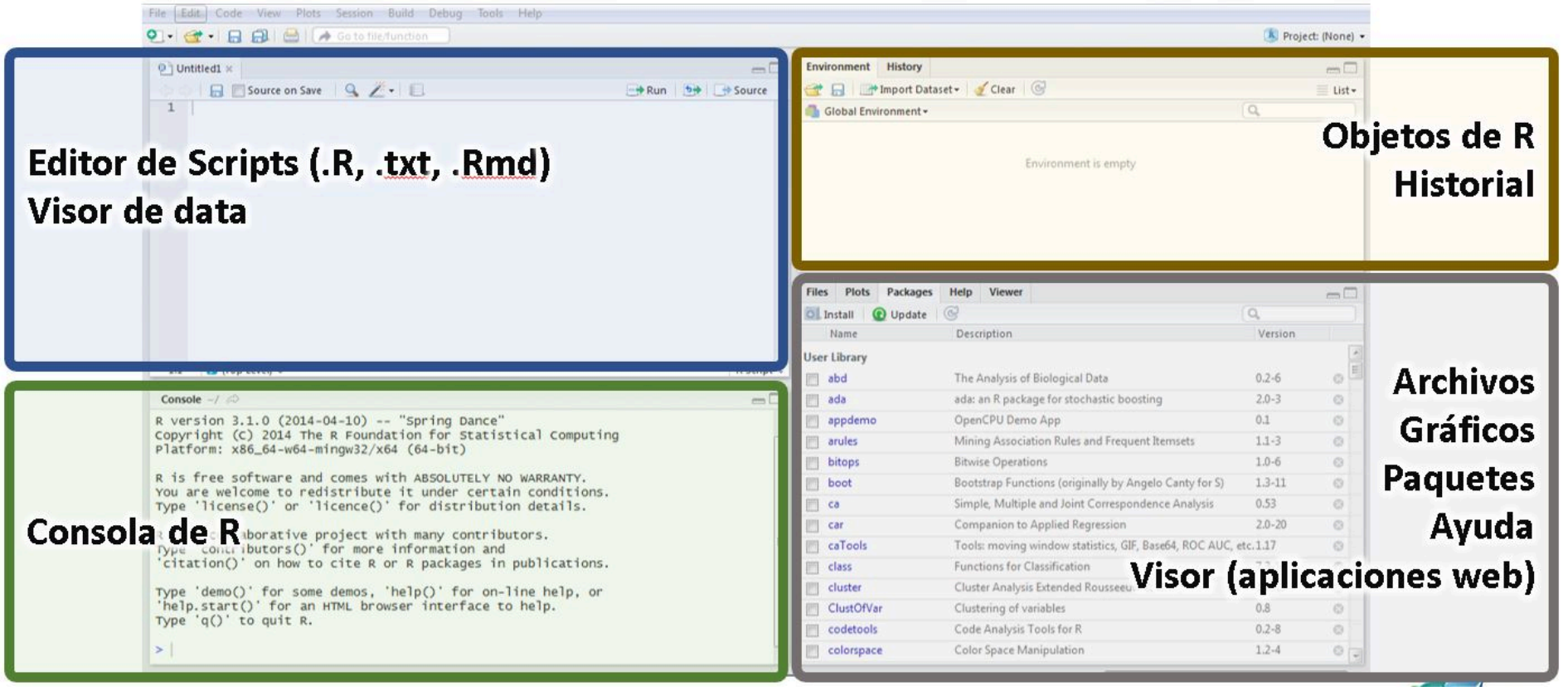
RStudio

Pantalla inicial de RStudio



RStudio

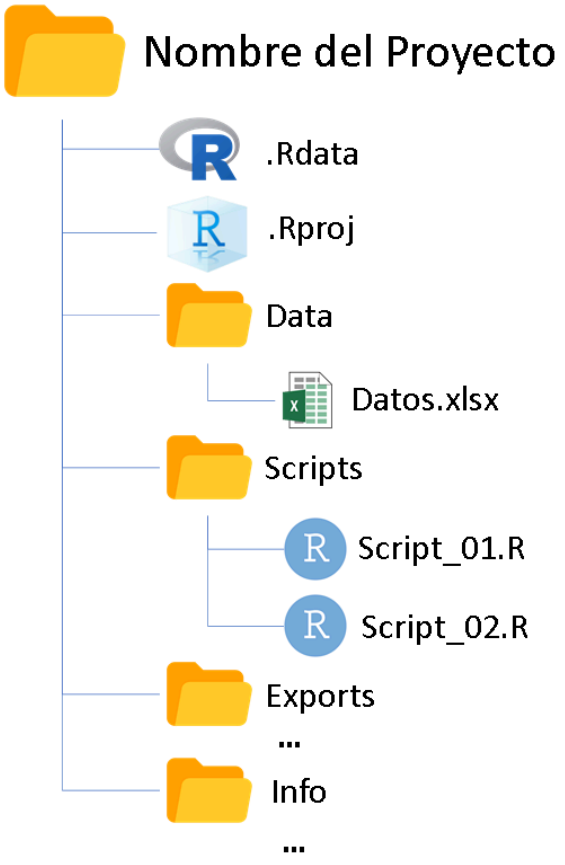
Interfaz de RStudio se divide en 4 paneles:



Proyectos en R, RStudio

- Un **Proyecto** es una carpeta que contiene todos los scripts, los archivos desde donde se importan los datos y los archivos de proyecto como el .RData (que contiene los objetos con los que se está trabajando) y .Rhistory (que contiene la historia de comandos ejecutados)
- Permite tener nuestros análisis ordenados,
- Al abrir un proyecto antiguo RStudio lo abre con las pestañas que se tenía activas,
- Permite colaboración utilizando GIT o Subversion,
- Se sugiere tener una estructura interior, por ejemplo: Scripts, Data, Exports, Info
- En R podemos hacer:

```
dir.create('Scripts')
dir.create('Data')
dir.create('Exports')
```



Realizar un script en RStudio

Acción	Atajo
Nuevo script	`Ctrl + Shift + N`
Ejecutar línea o selección	`Ctrl + Enter`
Insertar <-	`Alt + -`
Comentar / Des-comentar	`Ctrl + Shift + C`
Ir al Source Editor	`Ctrl + 1`
Ir a la Consola	`Ctrl + 2`
Crear sección de código	`Ctrl + Shift + R`
Completado de código	`Tab`
Saltar a función o sección	`Alt + Shift + J`
Reformatear línea	`Ctrl + I`

Generalidades

- R, aparte de objetos, tiene:
  - Expresión.- Se evalúa, se imprime y el valor se pierde

```
5+5 # Expresión
```

```
## [1] 10
```

- Asignación.- Evalúa la expresión y guarda el resultado en una variable (no lo imprime)

```
a <- 5+5 # Asigna el valor a la variable "a"
```

⚠️ R es case sensitivity (`Abc` es diferente de `abc`)

R como calculadora

```
2 + 3*5
```

```
## [1] 17
```

```
log((1+2+3)/4) # log natural
```

```
## [1] 0.4054651
```

```
pi^2 # pi y potencia
```

```
## [1] 9.869604
```

> Notar que todo esto son expresiones porque se calculan y se muestran, no se guarda el resultado en ningún objeto de R

R como calculadora

```
abs(-2) # valor absoluto
```

```
## [1] 2
```

```
factorial(3) # factorial
```

```
## [1] 6
```

```
floor(5.7) # funcion piso
```

```
## [1] 5
```

> Notar que todo esto son expresiones porque se calculan y se muestran, no se guarda el resultado en ningún objeto de R

Generar secuencias, repeticiones y aleatorios

```
1:10 # secuencia de 1 a 10, de 1 en 1
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

```
seq(from= 0, to= 20, by= 5) # función seq
```

```
## [1]  0  5 10 15 20
```

```
seq(from= 5, by= 5, length.out= 5) # función seq
```

```
## [1]  5 10 15 20 25
```

> Notar que todo esto son expresiones porque se calculan y se muestran, no se guarda el resultado en ningún objeto de R

Manejo de paquetes: conceptos clave

En R, un **paquete** es una colección de funciones, datos y documentación que extiende las capacidades base. El ciclo de vida de un paquete tiene dos pasos claramente distintos:

1. Instalar *(una sola vez por máquina)*

Descarga el paquete desde CRAN y lo guarda en disco.

```
install.packages('nombre_paquete')
```

Es como comprar e instalar un programa.
No es necesario repetirlo salvo que actualices R.

2. Cargar *(cada vez que abres R/RStudio)*

```
library('nombre_paquete')
```

Pone el paquete disponible en la sesión actual (en la RAM).
Es como abrir el programa para usarlo.

⚠️ **Error frecuente:** instalar el paquete pero olvidar cargarlo con `library()`.
R no reconocerá ninguna de sus funciones.

💡 Coloca todos tus `library()` al **inicio** del script, antes de cualquier otra línea de código. Así es claro qué paquetes necesita el análisis.

Manejo de paquetes: comandos útiles

Gestión desde la consola

Acción	Comando
Instalar	`install.packages('pkg')`
Instalar varios	`install.packages(c('pkg1','pkg2'))`
Cargar	`library('pkg')`
Ver si está instalado	`require('pkg')`
Descargar de la sesión	`detach('package:pkg')`
Ver paquetes cargados	`search()`
Ver todos los instalados	`installed.packages()`
Actualizar paquetes	`update.packages()`

Gestión visual desde RStudio

La pestaña **Packages** de RStudio permite instalar, cargar, actualizar y ver la documentación sin escribir código.

Instalar desde GitHub (versiones de desarrollo)

```
# Requiere el paquete remotes
install.packages('remotes')
remotes::install_github('usuario/repo')
```

Usar una función sin cargar el paquete
¿Y si no quiero cargar todo el paquete porque sólo voy a usar una vez una función? Entonces se puede hacer:

```
# Con el operador :: no hace falta library()
readxl::read_excel('archivo.xlsx')
```

Paquetes a usar (primeros)

Instalar los paquetes (esto se ejecuta una sóla vez)

```
install.packages("magrittr") # pipe
install.packages("tidyverse") # Manipulación de datos y ggplot2
install.packages("readxl")   # Importar Excel
```

Paquetes a usar (primeros)

Activar los paquetes (esto se ejecutará cada vez que se abra el proyecto o R)

```
# Cargar la libreria a utilizar
library(tidyverse) # Manipulación de datos y ggplot2
```

```
## Warning: package 'tidyverse' was built under R version 4.3.3
```

```
## Warning: package 'dplyr' was built under R version 4.3.3
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ dplyr     1.1.4     ✓ readr     2.1.5
## ✓ forcats   1.0.0     ✓ stringr   1.5.1
## ✓ ggplot2   4.0.3     ✓ tibble    3.2.1
## ✓ lubridate 1.9.3     ✓ tidyr     1.3.1
## ✓ purrr     1.0.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(magrittr)  # pipe
```

```
## Warning: package 'magrittr' was built under R version 4.3.3
```

```
##
```

El Tidyverse: ecosistema completo

Paquete	Función
`readr`	Leer archivos de texto
`readxl`	Leer Excel
`dplyr`	Manipular datos
`tidyr`	Ordenar (pivot, unnest...)
`ggplot2`	Visualización
`stringr`	Manipular texto
`forcats`	Manipular factores
`lubridate`	Manipular fechas

Filosofía tidy data:

- Cada fila = una observación
- Cada columna = una variable
- Cada celda = un valor

```
# Todo el tidyverse con un solo comando:
library(tidyverse)
```

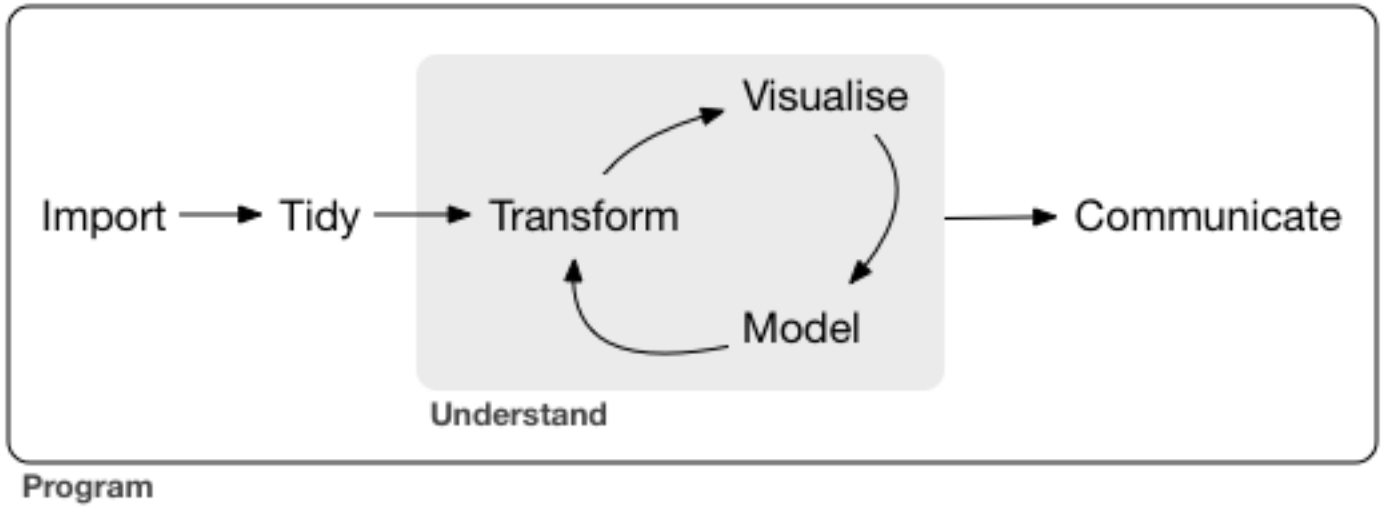
Más info: tidyverse.org

Desarrollo de un Caso de Análisis de Datos

Curso: R-Evolution: Análisis de Datos con R+IA

Néstor Montaño P.

Workflow de un análisis estadístico



- **Import**: Obtener y entender los datos
- **Tidy**: Ordenar los datos de tal manera que sea sencillo transformarlos y visualizarlos
- **Transform**: Manipular los datos hasta obtener el input que el análisis necesita
- **Visualise**: Realizar el análisis exploratorio de datos
- **Model**: Aplicar técnicas estadísticas para el entendimiento del problema
- **Communicate**: Mostrar los resultados: reportes, gráficos, dashboards, web apps

Ejemplo: Data de transacciones bancarias

El Banco del Pacífico requiere mejorar los tiempos de atención al cliente en ventanilla, para ello ha recolectado esta información anónimamente para cada cajero y transacción realizada.

Le suministran un excel con:

1. **Hoja Data**: columnas Sucursal, Cajero, ID_Transaccion, Transaccion, Tiempo_Servicio_seg, Satisfaccion, Monto
2. **Hoja Data_Sucursal**: indica si en la sucursal se ha puesto o no el nuevo sistema
3. **Datos demográficos** de los cajeros

Ejemplo - Importar

```
# Leer el archivo de excel y asignarlo al objeto data_banco
data_banco    <- readxl::read_excel("Data/Data_Banco.xlsx", sheet = "Data")
data_sucursal <- readxl::read_excel("Data/Data_Banco.xlsx", sheet = "Data_Sucursal")
```

Ejemplo - Importar

Bien, se han creado dos objetos en nuestro 'environment', **¿qué tipo de estructura hemos importado?** R. Un data.frame

- Se puede ver la estructura de un objeto con `str()`

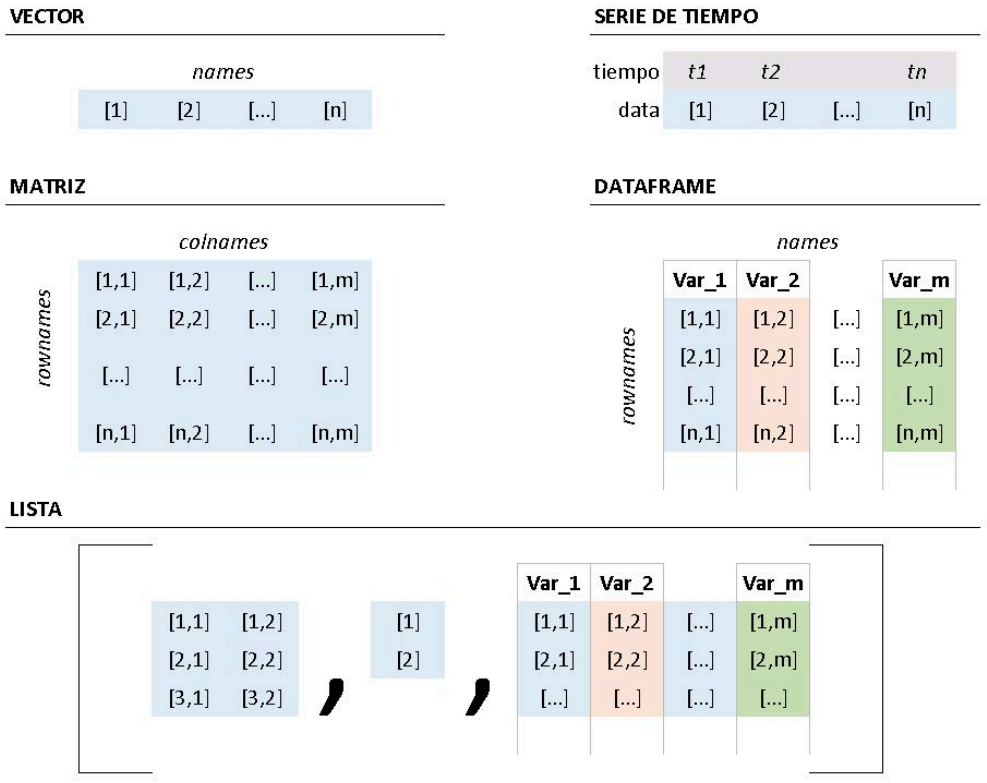
```
str(data_banco)
```

```
## tibble [24,299 × 7] (S3: tbl_df/tbl/data.frame)
##  $ Sucursal           : num [1:24299] 62 62 62 62 62 62 62 62 62 62 ...
##  $ Cajero             : num [1:24299] 4820 4820 4820 4820 4820 4820 4820 4820 4820 4820 ...
##  $ ID_Transaccion     : chr [1:24299] "2" "2" "2" "2" ...
##  $ Transaccion        : chr [1:24299] "Cobro/Pago (Cta externa)" "Cobro/Pago (Cta externa)" "Cobro/P
##  $ Tiempo_Servicio_seg: num [1:24299] 622 329 643 422 378 487 384 910 694 650 ...
##  $ Satisfaccion       : chr [1:24299] "Muy Bueno" "Malo" "Regular" "Regular" ...
##  $ Monto              : chr [1:24299] "2889,3" "1670,69" "3172,49" "1764.92" ...
```

Estructuras de datos en R

R cuenta con un sinnúmero de estructuras de datos (clases de objetos), los más usados son:

- **Vector**: colección ordenada de elementos del mismo tipo
- **Matriz**: vector bidimensional
- **Data.frame**: tabla con columnas de distintos tipos (como Excel)
- **Tibble**: versión mejorada del data.frame (tidyverse)
- **Lista**: colección de objetos de cualquier tipo
- **Serie de Tiempo**: vector con información temporal



Data.frames

- Data.frame (y tibble) es una lista de vectores, cumple:
  - Las componentes son vectores
  - Cada vector puede ser de un tipo de dato distinto
  - Cada elemento/columna es una variable
  - Las columnas tienen el mismo largo
- Se podría decir que un data.frame es como una tabla en una hoja de excel

```
head(data_banco)
```

```
## # A tibble: 6 × 7
##   Sucursal Cajero ID_Transaccion Transaccion    Tiempo_Servicio_seg Satisfaccion
##      <dbl>  <dbl> <chr>          <chr>                       <dbl> <chr>
## 1       62   4820 2              Cobro/Pago (C…                622 Muy Bueno
## 2       62   4820 2              Cobro/Pago (C…                329 Malo
## 3       62   4820 2              Cobro/Pago (C…                643 Regular
## 4       62   4820 2              Cobro/Pago (C…                422 Regular
## 5       62   4820 2              Cobro/Pago (C…                378 Muy Bueno
## 6       62   4820 2              Cobro/Pago (C…                487 Bueno
## # ℹ 1 more variable: Monto <chr>
```

Entender los datos

Luego de importar se debe entender los datos

- ¿Qué representa cada columna?
- ¿Qué tipo de dato debería tener cada columna?
- ¿Qué granularidad o atomicidad tiene la data?
- Si se tiene varios conjuntos de datos ¿Cómo se relacionan?
- ¿A qué periodo de tiempo corresponde la data?
- Muchas veces se obtiene la información desde una base de datos y por tanto toca entender la base y el query que genera los datos

Ejemplo - Entender los datos

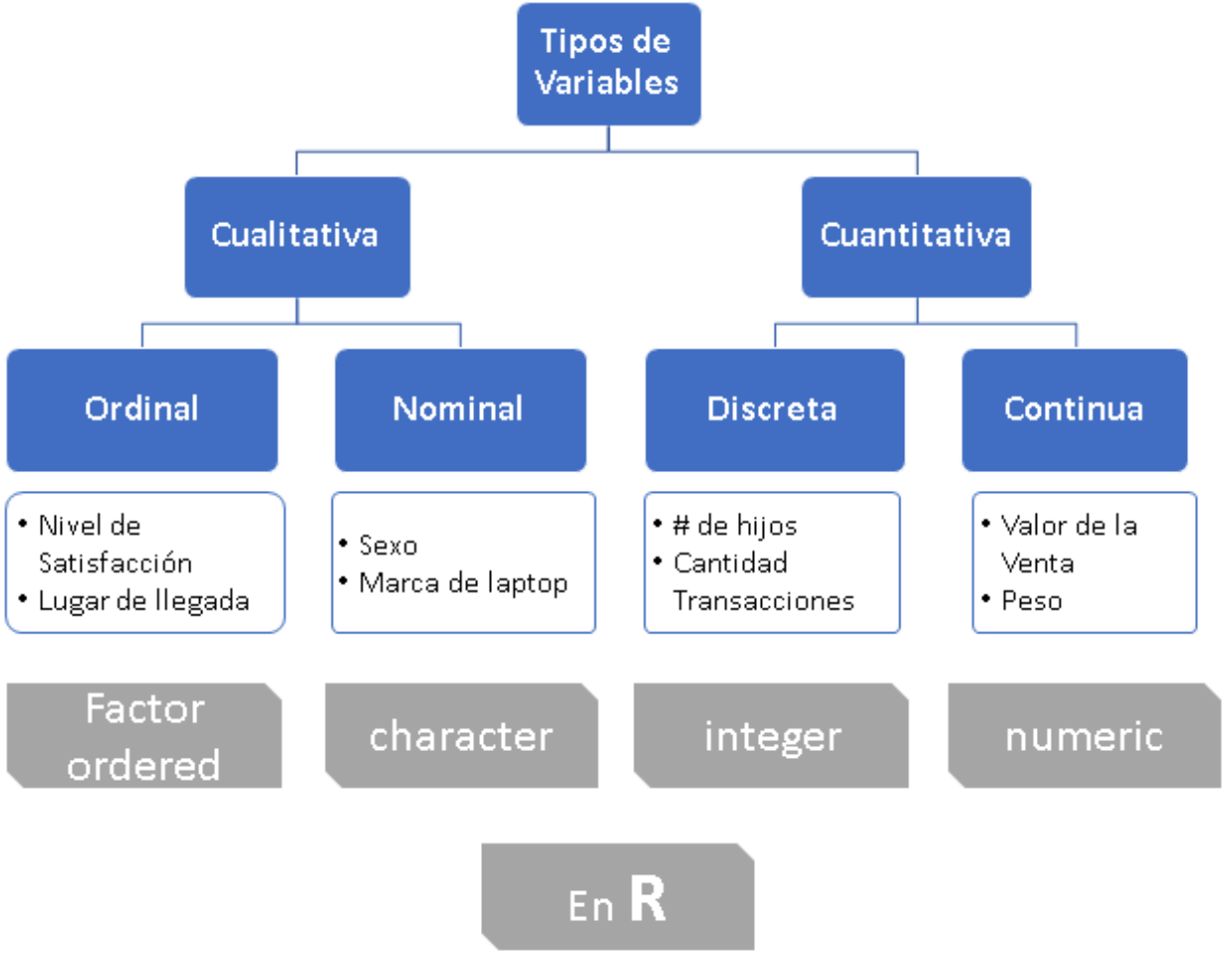
`glimpse` permite dar un vistazo rápido al dataframe, nos da los nombres de las columnas, los tipos de datos y una muestra de sus valores

```
glimpse(data_banco)
```

```
## Rows: 24,299
## Columns: 7
## $ Sucursal            <dbl> 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62, 62…
## $ Cajero              <dbl> 4820, 4820, 4820, 4820, 4820, 4820, 4820, 4820, 48…
## $ ID_Transaccion      <chr> "2", "2", "2", "2", "2", "2", "2", "2", "2", "2", …
## $ Transaccion         <chr> "Cobro/Pago (Cta externa)", "Cobro/Pago (Cta exter…
## $ Tiempo_Servicio_seg <dbl> 622, 329, 643, 422, 378, 487, 384, 910, 694, 650, …
## $ Satisfaccion        <chr> "Muy Bueno", "Malo", "Regular", "Regular", "Muy Bu…
## $ Monto               <chr> "2889,3", "1670,69", "3172,49", "1764.92", "1835.6…
```

Tipos de variables

Tipos de variables y su correspondencia en R



Manipulacion de datos - Básico

Curso: R-Evolution: Análisis de Datos con R+IA

Néstor Montaño P.

Manipulacion de datos

R tiene sus comandos predeterminados para manipular datos (*R Base*), sin embargo existen varios paquetes que simplifican esta tarea. En este curso veremos como hacerlo con el paquete **dplyr** (y *magrittr*) que están dentro del conjunto de paquetes llamado **tidyverse**

Paquete	Función
`readr`	Leer archivos de texto
`readxl`	Leer Excel
`dplyr`	Manipular datos
`tidyr`	Ordenar (pivot, unnest...)
`ggplot2`	Visualización
`stringr`	Manipular texto
`forcats`	Manipular factores
`lubridate`	Manipular fechas

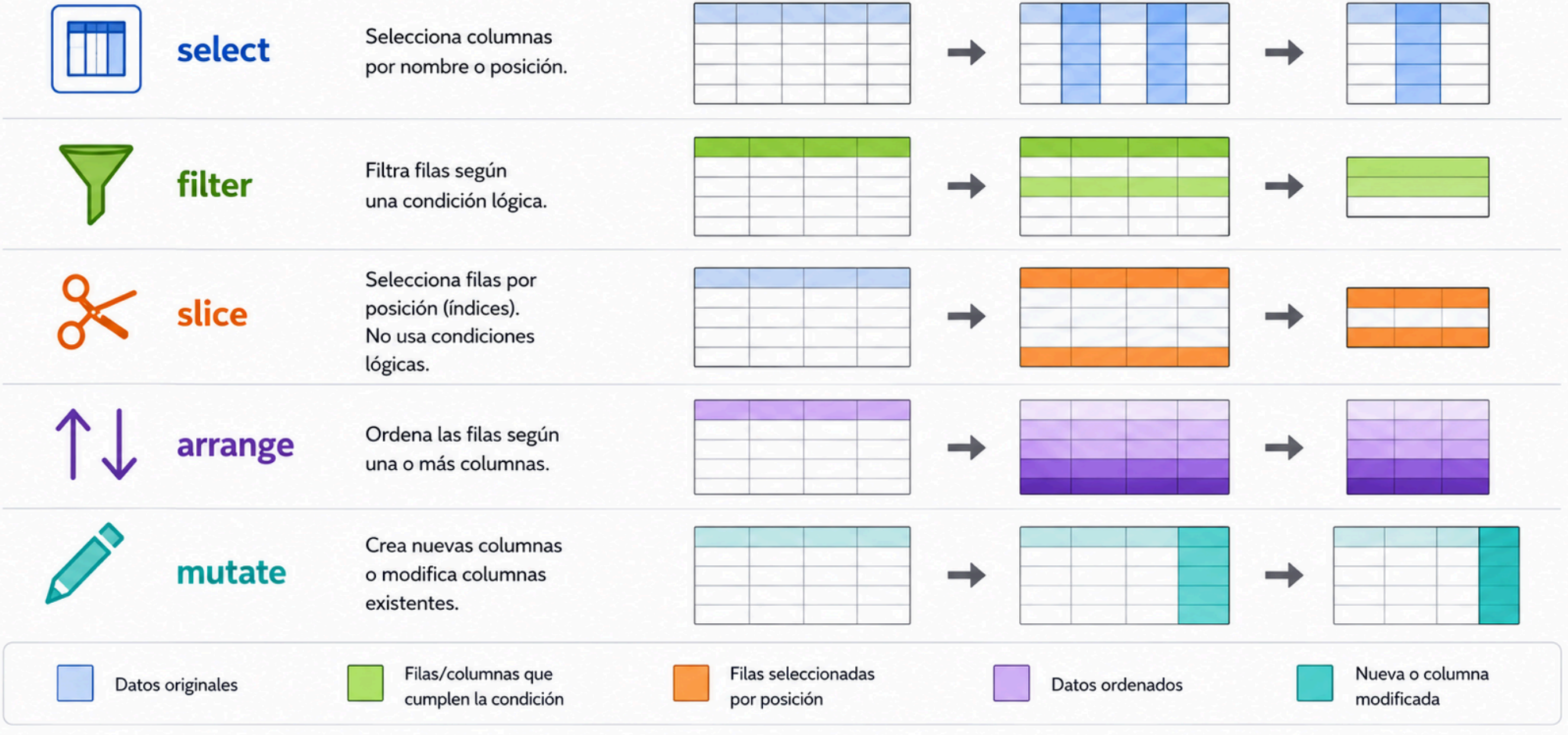
Filosofía tidy data:

- Cada fila = una observación
- Cada columna = una variable
- Cada celda = un valor

```
# Todo el tidyverse con un solo comando:
library(tidyverse)
```

Más info: tidyverse.org

dplyr: verbos principales



Seleccionar columnas: select()

Seleccionar las columnas Transaccion, Tiempo_Servicio_seg del data.frame data_banco pero usando %>%

```
# Se lee: del data_banco, selecciona las columnas Transaccion y Tiempo_Servicio_seg
# data_banco[ , c("Transaccion", "Tiempo_Servicio_seg") ] ## Base de R
data_banco %>% select(Transaccion, Tiempo_Servicio_seg)
```

```
## # A tibble: 24,299 × 2
##    Transaccion                Tiempo_Servicio_seg
##    <chr>                                    <dbl>
##  1 Cobro/Pago (Cta externa)                   622
##  2 Cobro/Pago (Cta externa)                   329
##  3 Cobro/Pago (Cta externa)                   643
##  4 Cobro/Pago (Cta externa)                   422
##  5 Cobro/Pago (Cta externa)                   378
##  6 Cobro/Pago (Cta externa)                   487
##  7 Cobro/Pago (Cta externa)                   384
##  8 Cobro/Pago (Cta externa)                   910
##  9 Cobro/Pago (Cta externa)                   694
## 10 Cobro/Pago (Cta externa)                   650
## # ℹ 24,289 more rows
```

Seleccionar columnas: select()

Seleccionar según nombre de la columna/variable.

```
# Seleccionar todas las columnas menos Cajero
data_banco %>% select(-Cajero)
# Seleccionar todas las columnas cuyo nombre contenga el texto "Tra"
data_banco %>% select(contains("Tra"))
# Mostrar el resultado en el visor de RStudio
data_banco %>% select(contains("Tra"))
# Seleccionar todas las columnas cuyo nombre inicie con "S"
data_banco %>% select(starts_with("S"))
# Seleccionar todas las columnas cuyo nombre finalice con "on"
data_banco %>% select(ends_with("on"))
# Seleccionar todas las columnas cuyo nombre contenga una "r" o un "sa"
data_banco %>% select(matches("r?sa"))
# Más información sobre expresiones regulares usando: ?base::regex
```

Filtrar/Seleccionar filas: filter() y slice()

Para filtrar las filas según la posición de la fila se usa `slice()`

```
data_banco %>% slice(3:6)
```

```
## # A tibble: 4 × 7
##   Sucursal Cajero ID_Transaccion Transaccion    Tiempo_Servicio_seg Satisfaccion
##      <dbl>  <dbl> <chr>          <chr>                       <dbl> <chr>
## 1       62   4820 2              Cobro/Pago (C…                643 Regular
## 2       62   4820 2              Cobro/Pago (C…                422 Regular
## 3       62   4820 2              Cobro/Pago (C…                378 Muy Bueno
## 4       62   4820 2              Cobro/Pago (C…                487 Bueno
## # ℹ 1 more variable: Monto <chr>
```

Existe también `slice_sample()`, `slice_min()`, `slice_max()` que permiten seleccionar una muestra o la fila con el menor o mayor valor respectivamente

```
# Seleccionar las dos filas con menor valor en TIempo de servicio
data_banco %>% slice_min(Tiempo_Servicio_seg, n = 2)
```

```
## # A tibble: 2 × 7
##   Sucursal Cajero ID_Transaccion Transaccion Tiempo_Servicio_seg Satisfaccion
##      <dbl>  <dbl> <chr>          <chr>                    <dbl> <chr>
## 1       85    472 3              Deposito                    18 Regular
## 2       85    472 3              Deposito                    18 Malo
## # ℹ 1 more variable: Monto <chr>
```

Filtrar/Seleccionar filas: filter()

Filtrar las filas según las condiciones dadas en filter()

```
# Filtrar las filas correspondientes a la sucursal 62
data_banco %>% filter(Sucursal == 62)
# ver el resultado en el visor de RStudio
data_banco %>% filter(Sucursal == 62) %>% View
# Filtrar las filas correspondientes a la sucursal 62 y hayan durado más de 120 seg
data_banco %>% filter(Sucursal == 62 & Tiempo_Servicio_seg > 120) %>% View
# Filtrar las filas correspondientes a la sucursal 62, hayan durado
# más de 120 segundos y la evaluación a la satisfacción sea Muy Bueno
data_banco %>% filter(Sucursal == 62 & Tiempo_Servicio_seg > 120 &
                        Satisfaccion == "Muy Bueno") %>% View
```

Ordenar las filas: arrange()

Ordenar las filas según lo expresado en arrange()

```
# Ordenar por la satisfaccion
data_banco %>% arrange(Satisfaccion) %>% View
# Ordenar cada Transaccion y dentro de cada transaccion
# de mayor a menor por tiempo de servicio
data_banco %>% arrange(Transaccion, desc(Tiempo_Servicio_seg)) %>% View
```

Crear o modificar columnas/variables: mutate()

Crear una nueva columna con el tiempo en minutos

```
# Crear una nueva columna con el tiempo en minutos
data_banco %>% mutate(Tiempo_Servicio_Min= Tiempo_Servicio_seg/60)
```

```
## # A tibble: 24,299 × 8
##    Sucursal Cajero ID_Transaccion Transaccion   Tiempo_Servicio_seg Satisfaccion
##       <dbl>  <dbl> <chr>          <chr>                      <dbl> <chr>
##  1       62   4820 2              Cobro/Pago (…                622 Muy Bueno
##  2       62   4820 2              Cobro/Pago (…                329 Malo
##  3       62   4820 2              Cobro/Pago (…                643 Regular
##  4       62   4820 2              Cobro/Pago (…                422 Regular
##  5       62   4820 2              Cobro/Pago (…                378 Muy Bueno
##  6       62   4820 2              Cobro/Pago (…                487 Bueno
##  7       62   4820 2              Cobro/Pago (…                384 Regular
##  8       62   4820 2              Cobro/Pago (…                910 Bueno
##  9       62   4820 2              Cobro/Pago (…                694 Muy Bueno
## 10       62   4820 2              Cobro/Pago (…                650 Muy Bueno
## # ℹ 24,289 more rows
## # ℹ 2 more variables: Monto <chr>, Tiempo_Servicio_Min <dbl>
```

Nótese que **no se asignó**, el objeto data_banco no tiene la columna Tiempo_Servicio_Min

Crear o modificar columnas/variables: mutate()

Crear una nueva columna con el tiempo en minutos y **asignar** el resultado

```
# Crear una nueva columna con el tiempo en minutos
data_banco <- data_banco %>%
  mutate(Tiempo_Servicio_Min= Tiempo_Servicio_seg/60)
# Mostrar
data_banco
```

```
## # A tibble: 24,299 × 8
##    Sucursal Cajero ID_Transaccion Transaccion   Tiempo_Servicio_seg Satisfaccion
##       <dbl>  <dbl> <chr>          <chr>                      <dbl> <chr>
##  1       62   4820 2              Cobro/Pago (…                622 Muy Bueno
##  2       62   4820 2              Cobro/Pago (…                329 Malo
##  3       62   4820 2              Cobro/Pago (…                643 Regular
##  4       62   4820 2              Cobro/Pago (…                422 Regular
##  5       62   4820 2              Cobro/Pago (…                378 Muy Bueno
##  6       62   4820 2              Cobro/Pago (…                487 Bueno
##  7       62   4820 2              Cobro/Pago (…                384 Regular
##  8       62   4820 2              Cobro/Pago (…                910 Bueno
##  9       62   4820 2              Cobro/Pago (…                694 Muy Bueno
## 10       62   4820 2              Cobro/Pago (…                650 Muy Bueno
## # ℹ 24,289 more rows
## # ℹ 2 more variables: Monto <chr>, Tiempo_Servicio_Min <dbl>
```

Eliminar columnas/variables: select(!())

Para eliminar una columna, podemos simplemente seleccionar en el dataframe las columnas que queremos

```
# Eliminar la columna Tiempo_Servicio_Min
data_banco <- data_banco %>%
  select(!(Tiempo_Servicio_Min))
# Mostrar
data_banco
```

```
## # A tibble: 24,299 × 7
##    Sucursal Cajero ID_Transaccion Transaccion   Tiempo_Servicio_seg Satisfaccion
##       <dbl>  <dbl> <chr>          <chr>                      <dbl> <chr>
##  1       62   4820 2              Cobro/Pago (…                622 Muy Bueno
##  2       62   4820 2              Cobro/Pago (…                329 Malo
##  3       62   4820 2              Cobro/Pago (…                643 Regular
##  4       62   4820 2              Cobro/Pago (…                422 Regular
##  5       62   4820 2              Cobro/Pago (…                378 Muy Bueno
##  6       62   4820 2              Cobro/Pago (…                487 Bueno
##  7       62   4820 2              Cobro/Pago (…                384 Regular
##  8       62   4820 2              Cobro/Pago (…                910 Bueno
##  9       62   4820 2              Cobro/Pago (…                694 Muy Bueno
## 10       62   4820 2              Cobro/Pago (…                650 Muy Bueno
## # ℹ 24,289 more rows
## # ℹ 1 more variable: Monto <chr>
```

FIN

Curso: R-Evolution: Análisis de Datos con R+IA

Néstor Montaño P.